UNIVERSITY OF PISA

COMPUTER SCIENCE - ARTIFICIAL INTELLIGENCE

Project Track 19 — Non-ML

Computational Mathematics for Learning and Data Analysis Project

QR Factorization and Limited-Memory Quasi-Newton Methods for Regularized Least Squares

Student:
Karol Jinet Cardona Bolanos
Mat. 702023

Student:
Gianluca Panzani
Mat. 550358

ACADEMIC YEAR 2025/2026

Contents

Chapter 1

Introduction

This project addresses the numerical solution of a regularized linear least squares problem. Given a data matrix $X \in \mathbb{R}^{m \times n}$ from the ML-CUP dataset, a random response vector $y \in \mathbb{R}^n$, and a regularization parameter $\lambda > 0$, the goal is to find the vector $w \in \mathbb{R}^m$ that minimizes

$$\min_w \left\| \hat{X} w - \hat{y} \right\|, \tag{1.1}$$

where the augmented matrix and vector are defined as

$$\hat{X} = \begin{bmatrix} X^T \\ \lambda I_m \end{bmatrix} \in \mathbb{R}^{(n+m) \times m}, \qquad \hat{y} = \begin{bmatrix} y \\ 0 \end{bmatrix} \in \mathbb{R}^{n+m}. \tag{1.2}$$

Expanding the squared norm, the problem is equivalent to minimizing

$$f(w) = \frac{1}{2} \left\| X^T w - y \right\|^2 + \frac{1}{2} \lambda^2 \left\| w \right\|^2, \tag{1.3}$$

which is a standard instance of Tikhonov regularization (ridge regression). The regularization term $\lambda^2 \left\| w \right\|^2$ penalizes large coefficient vectors, improving the numerical stability of the solution and controlling overfitting.

Two computational strategies are explored:

- **(A1)** A limited-memory quasi-Newton method (L-BFGS), which iteratively approximates the curvature of the objective function using only gradient information and a small number of stored vector pairs.
- **(A2)** A direct method based on thin QR factorization with Householder reflectors, in the compact variant where the orthogonal matrix Q is not formed explicitly, but applied implicitly through stored Householder vectors.

Both algorithms are implemented from scratch, without relying on any off-the-shelf numerical solvers.

Chapter 2

Problem Properties

Before discussing the optimization algorithms, we analyze the mathematical properties of the objective function (1.3) that are relevant for convergence.

2.0.1 Gradient and Hessian

The objective (1.3) is a quadratic function of w. Consequently, its Hessian matrix is constant, as it does not depend on the variable w but only on the fixed data matrix X and the regularization parameter λ. Its gradient and Hessian are:

$$\nabla f(w) = X(X^T w - y) + \lambda^2 w, \tag{2.1}$$

$$H \coloneqq \nabla^2 f(w) = XX^T + \lambda^2 I_m. \tag{2.2}$$

2.0.2 Strong Convexity and Smoothness

To guarantee the existence of a unique global minimum and to analyze the convergence speed of our optimization algorithms, we first establish the strong convexity and smoothness of the objective function.

By Lemma 1, the Hessian $H = XX^T + \lambda^2 I_m$ is symmetric positive definite whenever $\lambda > 0$. This confirms that $f(w)$ is strictly convex, ensuring that any local minimum is also the unique global minimum.

Formally, this implies that f is μ-strongly convex and L-smooth, where the constants μ and L correspond respectively to the smallest and largest eigenvalues of H. By Lemma 2 (eigenvalue shift), the eigenvalues of $H = XX^T + \lambda^2 I_m$ are obtained by shifting those of XX^T by λ^2:

$$\mu = \lambda_{\min}(H) = \lambda_{\min}(XX^T) + \lambda^2, \qquad L = \lambda_{\max}(H) = \lambda_{\max}(XX^T) + \lambda^2. \tag{2.3}$$

The smoothness constant L bounds how rapidly the gradient can change. For all $w, w' \in \mathbb{R}^m$:

$$\left\|\nabla f(w) - \nabla f(w')\right\| = \left\|H(w - w')\right\| \leq \|H\| \left\|w - w'\right\| = L \left\|w - w'\right\|. \tag{2.4}$$

Since our data matrix $X \in \mathbb{R}^{m \times n}$ is tall-thin ($m > n$), by Lemma 3 the eigenvalues of XX^T are $\{\sigma_1^2, \ldots, \sigma_n^2, 0, \ldots, 0\}$, so in particular $\lambda_{\min}(XX^T) = 0$. Therefore, the strong convexity constant simplifies to:

$$\mu = \lambda^2. \tag{2.5}$$

This highlights the crucial role of λ: without regularization, H would be only positive *semi*-definite, $\mu = 0$, and f would lack strong convexity. In that case, the system $Hw = Xy$ would be underdetermined, the minimum would not be unique, and iterative methods would not converge to a well-defined solution.

Together, L and μ quantify the curvature of the objective: μ controls the minimum curvature (how sharply f rises in the flattest direction) and L the maximum curvature (how sharply it rises in the steepest direction). Their ratio defines the condition number of the problem, which directly impacts the convergence speed of iterative solvers such as L-BFGS.

2.0.3 Condition Number

The geometric shape of the objective function—and consequently the difficulty of the optimization problem—is summarized by the condition number of the augmented matrix $\hat{X}$, defined as the ratio of its largest to smallest singular values.

To express this in terms of the problem parameters, we note that the singular values of $\hat{X}$ are the square roots of the eigenvalues of $\hat{X}^T\hat{X}$. A direct computation gives:

$$\hat{X}^T\hat{X} = \begin{bmatrix} X & \lambda I_m \end{bmatrix} \begin{bmatrix} X^T \\ \lambda I_m \end{bmatrix} = XX^T + \lambda^2 I_m = H, \tag{2.6}$$

so $\sigma_i(\hat{X}) = \sqrt{\lambda_i(H)}$ for each i. Combining this with equations (2.3), the condition number of $\hat{X}$ is:

$$\kappa(\hat{X}) = \frac{\sigma_{\max}(\hat{X})}{\sigma_{\min}(\hat{X})} = \sqrt{\frac{\lambda_{\max}(H)}{\lambda_{\min}(H)}} = \sqrt{\frac{L}{\mu}} = \frac{\sqrt{\lambda_{\max}(XX^T) + \lambda^2}}{\lambda}. \tag{2.7}$$

As the equation shows, the condition number depends heavily on the regularization parameter. For small values of λ, it scales as $\mathcal{O}(1/\lambda)$: weak regularization produces a highly elongated, canyon-like objective landscape, causing iterative methods to take many small zigzagging steps. Conversely, a large λ drives $\kappa(\hat{X}) \to 1$, making the problem nearly spherical and easy to solve from any direction.

Chapter 3

Limited-Memory BFGS (L-BFGS)

3.0.1 Quasi-Newton Methods: Background

Quasi-Newton methods are iterative algorithms for unconstrained optimization that approximate the Newton direction without explicitly computing the true Hessian matrix [1]. Recall that the pure Newton step would be $p_k = -[\nabla^2 f(w_k)]^{-1}\nabla f(w_k)$: it accounts for the local curvature and converges quadratically near the solution, but requires computing and factorizing an $m \times m$ matrix at every iteration, which costs $O(m^3)$ and is infeasible for large m. Quasi-Newton methods replace the true inverse Hessian with an approximation $H_k \approx [\nabla^2 f(w_k)]^{-1}$ that is built cheaply from first-order information only. The search direction is then:

$$p_k = -H_k \nabla f(w_k), \tag{3.1}$$

and the iterate is updated as $w_{k+1} = w_k + \alpha_k p_k$, where $\alpha_k > 0$ is the step length determined by a line search.

The secant equation. After each step, the algorithm observes two key vectors: the parameter displacement $s_k = w_{k+1} - w_k$ and the gradient difference $y_k = \nabla f(w_{k+1}) - \nabla f(w_k)$. These encode curvature information: in one dimension, the second derivative can be approximated as $f''(x) \approx \Delta f'/\Delta x = y_k/s_k$. In $\mathbb{R}^m$, this generalizes to the requirement that the new approximation H_{k+1} maps the gradient change y_k to the step s_k, exactly as the true inverse Hessian would near a quadratic:

$$H_{k+1}\, y_k = s_k. \tag{3.2}$$

This is called the *secant condition*. Geometrically, it forces H_{k+1} to reproduce the correct curvature along the direction just explored. However, in dimension $m > 1$, equation (3.2) has infinitely many solutions: it constrains H_{k+1} only along y_k, leaving all orthogonal directions undetermined.

The BFGS update. The BFGS method resolves the non-uniqueness by solving a constrained optimization problem: among all symmetric positive definite matrices satisfying the secant condition, find the one closest to the previous approximation H_k in a weighted Frobenius norm. This *minimal-change* principle ensures that information accumulated in H_k is preserved as much as possible, and only updated in the directions where new curvature has been observed. The unique solution is:

$$H_{k+1} = V_k^T H_k V_k + \rho_k s_k s_k^T, \tag{3.3}$$

where

$$\rho_k = \frac{1}{y_k^T s_k}, \qquad V_k = I - \rho_k y_k s_k^T. \tag{3.4}$$

The structure of this update is highly deliberate. The scalar ρ_k normalizes the curvature so that $\rho_k(s_k^T y_k) = 1$. The matrix V_k acts as a projector that annihilates y_k: since $V_k y_k = y_k - \rho_k y_k(s_k^T y_k) = y_k - y_k = 0$, the first term in (3.3) contributes nothing along y_k. The second term $\rho_k s_k s_k^T$ then injects precisely the curvature needed to satisfy the secant condition. Indeed, multiplying (3.3) by y_k:

$$H_{k+1}y_k = V_k^T H_k \underbrace{(V_k y_k)}_{=0} + \rho_k s_k \underbrace{(s_k^T y_k)}_{=1/\rho_k} = s_k. \checkmark \tag{3.5}$$

While BFGS achieves superlinear convergence on strongly convex problems, it has a fundamental limitation: after k iterations, H_k is a dense $m \times m$ matrix. Storing it requires $O(m^2)$ memory, and updating it via formula (3.3) costs $O(m^2)$ per iteration. For large m, both become prohibitive.

3.0.2 The L-BFGS Algorithm

The Limited-Memory BFGS (L-BFGS) method addresses the memory limitation with a key observation: we never need to *store* H_k explicitly. What we actually need at each iteration is the matrix-vector product $H_k \nabla f_k$. If we expand the BFGS recurrence (3.3) backwards over the last $\bar{m}$ steps, starting from an initial approximation $H_k^0 = \gamma_k I$, the product $H_k \nabla f_k$ can be expressed as a sequence of inner products and vector additions involving only the stored pairs $\{s_i, y_i\}_{i=k-\bar{m}}^{k-1}$. This leads to the *two-loop recursion* (Algorithm 1), which computes $H_k \nabla f_k$ implicitly in $O(\bar{m}\, m)$ operations, without ever forming or storing H_k.

In practice, $\bar{m} \in [3, 20]$ pairs are retained: Nocedal and Wright [1, Section 9.1] note that modest values of $\bar{m}$ often give satisfactory results, since curvature information from earlier iterations becomes increasingly irrelevant as the iterates move away from those regions. At each iteration, the oldest pair is discarded and the newest pair (s_k, y_k) is added.

Initial Scaling of H_k^0

At each iteration, the initial Hessian approximation is set to $H_k^0 = \gamma_k I$ for some scalar $\gamma_k > 0$. Conceptually, this is the "blank slate" from which the two-loop recursion builds H_k by applying the last $\bar{m}$ BFGS updates. The choice of γ_k matters: if it is too small, the search direction p_k will be short and many small steps will be needed; if it is too large, the algorithm will overshoot. We implement three strategies.

Nocedal scaling (BB2). The default choice [1, Eq. 9.6] is

$$\gamma_k^{\mathrm{N}} = \frac{s_{k-1}^T y_{k-1}}{y_{k-1}^T y_{k-1}}. \tag{3.6}$$

This is the unique minimiser of $\|\gamma\, y_{k-1} - s_{k-1}\|^2$ over $\gamma > 0$, i.e. the best scalar approximation to the inverse Hessian along s_{k-1} in a least-squares sense: γ_k^{N} is the scalar γ such that $\gamma y_{k-1} \approx s_{k-1}$, mimicking the secant condition $H_k^{-1} y_{k-1} = s_{k-1}$. It ensures that the search direction p_k is well-scaled and that the unit step length $\alpha_k = 1$ is accepted in most Wolfe line-search iterations. We note that this formula is algebraically identical to the *BB2* step size of Barzilai and Borwein [2, Eq. (5)], which was derived independently in the context of gradient descent by solving the same least-squares approximation to the secant equation, with $\Delta x \leftrightarrow s_{k-1}$ and $\Delta g \leftrightarrow y_{k-1}$. This observation motivates us to also consider the companion BB1 step as an alternative scaling strategy for H_k^0.

BB1 scaling. The companion step size proposed by Barzilai and Borwein [2, Eq. (6)] is

$$\gamma_k^{\mathrm{BB1}} = \frac{s_{k-1}^T s_{k-1}}{s_{k-1}^T y_{k-1}}. \tag{3.7}$$

This is the minimiser of $\|\gamma^{-1} s_{k-1} - y_{k-1}\|^2$, i.e. the scalar γ such that $\gamma^{-1} s_{k-1} \approx y_{k-1} \approx H\, s_{k-1}$, yielding an estimate of the *average* eigenvalue of $\nabla^2 f$ (rather than its inverse). On ill-conditioned problems, BB1 tends to produce larger steps than BB2, which can accelerate convergence by escaping flat regions more quickly.

Safeguarded BB2. The Nocedal scaling (3.6) can become numerically unreliable near the solution: as $w_k \to w^*$, the gradient changes between successive iterates vanish, so $y_{k-1}^T y_{k-1} \to 0$ while $s_{k-1}^T y_{k-1}$ remains bounded away from zero, causing $\gamma_k^{\mathrm{N}} \to \infty$. An unbounded scaling factor renders the search direction p_k unreliable regardless of the quality of the two-loop recursion. To prevent this, we clip the scaling factor to a finite interval:

$$\gamma_k^{\mathrm{safe}} = \mathrm{clip}\big(\gamma_k^{\mathrm{N}},\ \gamma_{\min},\ \gamma_{\max}\big)\,, \tag{3.8}$$

with defaults $\gamma_{\min} = 10^{-10}$ and $\gamma_{\max} = 10^{10}$. This retains the BB2 estimate in the well-behaved regime while preventing numerical overflow in degenerate cases.

Two-Loop Recursion

The product $r = H_k \nabla f_k$ is computed by Algorithm 1 [1, Algorithm 9.1]. The algorithm implements the BFGS recurrence (3.3) unrolled over the last $\bar{m}$ pairs, applied implicitly to the gradient vector q. The *backward loop* computes the α_i scalars that will be needed later, while simultaneously applying the V_i^T factors to q from right to left. The *scaling step* applies the base approximation $H_k^0 = \gamma_k I$. The *forward loop* then applies the rank-one corrections $\rho_i s_i s_i^T$, using the α_i computed in the backward pass. Together, the two loops implement exactly the product $H_k q$ without ever constructing the matrix.

Algorithm 1 L-BFGS Two-Loop Recursion [1, Algorithm 9.1]

Require: Gradient $q \leftarrow \nabla f_k$; stored pairs $\{s_i, y_i\}_{i=k-\bar{m}}^{k-1}$; scaling γ_k
Ensure: $r = H_k \nabla f_k$

1: **for** $i = k-1, k-2, \ldots, k-\bar{m}$ **do**
2: $\quad \rho_i \leftarrow 1/(y_i^T s_i)$; $\quad \alpha_i \leftarrow \rho_i\, s_i^T q$; $\quad q \leftarrow q - \alpha_i\, y_i$ ▷ Apply V_i^T to q
3: **end for**
4: $r \leftarrow \gamma_k\, q$ ▷ Apply $H_k^0 = \gamma_k I$
5: **for** $i = k-\bar{m}, \ldots, k-1$ **do**
6: $\quad \beta \leftarrow \rho_i\, y_i^T r$; $\quad r \leftarrow r + s_i\,(\alpha_i - \beta)$ ▷ Apply rank-one correction
7: **end for**
8: **return** r

The cost of Algorithm 1 is $4\bar{m}$ dot products of length m (two in each loop iteration) plus m multiplications for the scaling step, totalling $(4\bar{m}+1)\,m$ floating-point operations. This is a dramatic improvement over the $O(m^2)$ cost of storing and applying the full BFGS matrix.

Curvature Safeguard and Automatic Restart

For the BFGS formula (3.3) to produce a positive definite update, the *curvature condition* $y_k^T s_k > 0$ must hold. To see why, note that $\rho_k = 1/(y_k^T s_k)$ appears in the rank-one term $\rho_k s_k s_k^T$: if $y_k^T s_k \leq 0$, then $\rho_k \leq 0$ and the update would subtract from H_k, potentially making H_{k+1} indefinite and the next direction p_{k+1} a direction of ascent. We implement two levels of protection against this.

Negative-curvature reset. If $y_k^T s_k \leq 10^{-16}$, the entire stored memory is cleared and the algorithm restarts from a pure gradient-descent step (effectively setting $H_k = I$). This safeguard ensures that $\rho_k = 1/(y_k^T s_k)$ remains well-defined and positive, as required by the BFGS update (3.3). The threshold 10^{-16} corresponds to the unit roundoff in double precision arithmetic, below which ρ_k is no longer numerically reliable.

Curvature-based restart. As an additional safeguard, we clear the memory even when $y_k^T s_k > 0$ if the curvature estimate captured by the new pair changes dramatically relative to the previous one. To detect this, we track the quantity

$$\gamma_k = \frac{y_k^T s_k}{y_k^T y_k}, \tag{3.9}$$

which is the same as the Nocedal scaling (3.6): it estimates the effective inverse-Hessian scale along s_k. A sudden drop,

$$\gamma_k < \xi\, \gamma_{k-1}, \qquad \xi \in (0, 1), \tag{3.10}$$

signals that the curvature along the new step is much smaller than along the previous one. This typically happens when the iterates have moved into a qualitatively different region of the landscape: the pairs stored in memory were computed under a different curvature regime and now give a misleading approximation of H_k^{-1}. Clearing the memory and restarting from $H_k^0 = \gamma_k I$ is then preferable to letting corrupted pairs contaminate the two-loop recursion. The threshold $\xi = 0.2$ is used by default.

3.0.3 Line Search Strategies

Strong Wolfe Conditions

Once the descent direction p_k is computed, the step length α_k must be chosen to make sufficient progress without overshoot. The standard criterion for quasi-Newton methods is the *strong Wolfe conditions* [1, Eq. 3.7]:

$$f(w_k + \alpha_k p_k) \leq f(w_k) + c_1\, \alpha_k\, \nabla f_k^T p_k, \tag{3.11}$$

$$\left|\nabla f(w_k + \alpha_k p_k)^T p_k\right| \leq c_2\, \left|\nabla f_k^T p_k\right|. \tag{3.12}$$

Condition (3.11) (Armijo) ensures sufficient decrease: the function must drop by at least a fraction c_1 of the directional derivative. Without it, the algorithm might take steps so small that progress is negligible. Condition (3.12) prevents the step from being too short: the directional derivative at the new point must be small enough in absolute value, which ensures that the new pair (s_k, y_k) carries meaningful curvature information (in particular, it guarantees $y_k^T s_k > 0$). Following standard practice for quasi-Newton methods [1], we set $c_1 = 10^{-4}$ and $c_2 = 0.9$. Our implementation uses a bracketing-and-zoom procedure with cubic interpolation [1, Section 3.5].

Exact Line Search for Quadratic Objectives

The Wolfe conditions are designed for general nonlinear objectives and accept a range of step lengths rather than requiring the exact minimiser. For our quadratic f, we can do much better. The restriction of f along any direction p is a one-dimensional parabola:

$$\varphi(\alpha) = f(w + \alpha\, p) = f(w) + \alpha\, \nabla f(w)^T p + \frac{\alpha^2}{2}\, p^T H p. \tag{3.13}$$

Since $H \succ 0$, the coefficient $p^T H p > 0$ for all $p \neq 0$, so φ is a strictly convex parabola with a unique global minimum at $\varphi'(\alpha^*) = 0$, i.e.:

$$\alpha^* = -\frac{\nabla f(w)^T p}{p^T H p}. \tag{3.14}$$

For our specific Hessian $H = XX^T + \lambda^2 I_m$, the denominator can be expanded without forming H explicitly:

$$p^T H p = p^T (XX^T + \lambda^2 I_m)\, p = \left\| X^T p \right\|^2 + \lambda^2 \left\| p \right\|^2, \tag{3.15}$$

so the complete formula becomes a generalisation of [1, Eq. 3.25] to an arbitrary descent direction:

$$\alpha^* = -\frac{\nabla f(w)^T p}{\left\| X^T p \right\|^2 + \lambda^2 \left\| p \right\|^2}. \tag{3.16}$$

The only non-trivial cost is the single matrix-vector product $X^T p \in O(mn)$, identical to the cost of one gradient evaluation, and far cheaper than the multiple function and gradient evaluations required by the Wolfe bracketing procedure. Since α^* is the *exact* minimiser of φ, it fully exploits the parabolic structure and yields the best possible step at each iteration, reducing the number of iterations required to reach a given accuracy compared to the Wolfe variant.

3.0.4 Complete Algorithm

Algorithm 2 Enhanced L-BFGS [1, Algorithm 9.2]

Require: Starting point w_0; memory $\bar{m}$; tolerance ε; scaling strategy $\in$ {BB2, BB1, safe}; restart flag; restart threshold ξ

$k \leftarrow 0$; compute ∇f_0; $\tau \leftarrow \varepsilon \left\| \nabla f_0 \right\|$; $\gamma_{\text{prev}} \leftarrow 1$
while $\left\| \nabla f_k \right\| > \tau$ **do**
    Compute γ_k via chosen scaling strategy (Eq. (3.6), (3.7), or (3.8))
    $p_k \leftarrow -H_k \nabla f_k$ ▷ Via Algorithm 1 with $H_k^0 = \gamma_k I$
    if $\nabla f_k^T p_k \geq 0$ **then**
        $p_k \leftarrow -\nabla f_k$ ▷ Descent safeguard: fall back to steepest descent
    end if
    Compute α_k via exact LS (3.16) or Wolfe conditions (3.11)–(3.12)
    $w_{k+1} \leftarrow w_k + \alpha_k p_k$; $s_k \leftarrow \alpha_k p_k$; $y_k \leftarrow \nabla f_{k+1} - \nabla f_k$
    $\gamma_k \leftarrow (y_k^T s_k)/(y_k^T y_k)$
    if $y_k^T s_k > 10^{-10}$ **then**
        if restart enabled **and** $\gamma_k < \xi\, \gamma_{\text{prev}}$ **then**
            Clear all stored pairs ▷ Curvature-based restart, Eq. (3.10)
        end if
        Store (s_k, y_k); discard oldest if #stored $> \bar{m}$; $\gamma_{\text{prev}} \leftarrow \gamma_k$

else if $y_k^T s_k \leq 10^{-16}$ **then**
    Clear all stored pairs ▷ Negative-curvature reset
else
    Discard (s_k, y_k) ▷ Curvature too small to be reliable; pair skipped
end if
$k \leftarrow k + 1$
end while
return w_k

3.0.5 Stopping Criterion

The primary stopping criterion uses a *relative* gradient tolerance:

$$\|\nabla f(w_k)\| \leq \varepsilon \cdot \|\nabla f(w_0)\|. \tag{3.17}$$

The relative form is preferred over an absolute threshold $\|\nabla f(w_k)\| \leq \varepsilon$ because it is scale-invariant: the magnitude of ∇f depends on the problem scale (and in particular on λ), so an absolute threshold that is appropriate for one value of λ may be either too loose or too tight for another. Dividing by $\|\nabla f(w_0)\|$ normalizes for this effect.

As a secondary safeguard, the algorithm detects *numerical stagnation*: if the relative change $|f_k - f_{k-1}|/|f_k|$ drops below machine epsilon (approximately 10^{-16}) while the step length $\alpha_k \approx 0$, further progress is impossible due to floating-point limitations, and the iteration is terminated early.

3.0.6 Convergence Analysis

Global Linear Convergence

The convergence of L-BFGS on our problem follows from [3, Theorem 7.1]. That theorem requires three conditions, which we verify directly using the properties established in Section 2:

1. $f \in C^2$: satisfied, since f is quadratic;
2. the sublevel set $\mathcal{L}_0 = \{w : f(w) \leq f(w_0)\}$ is bounded: satisfied, since f is μ-strongly convex with $\mu = \lambda^2 > 0$, so all sublevel sets are compact ellipsoids;
3. there exist $M_1, M_2 > 0$ such that $M_1\|v\|^2 \leq v^T \nabla^2 f(w)\, v \leq M_2\|v\|^2$ for all v: satisfied with $M_1 = \mu = \lambda^2$ and $M_2 = L = \lambda_{\max}(XX^T) + \lambda^2$, since $\nabla^2 f = H$ is constant.

Under these conditions, the iterates satisfy the linear convergence bound:

$$f(w_k) - f(w^*) \leq r^k \left(f(w_0) - f(w^*)\right), \quad r \in [0, 1), \tag{3.18}$$

where the rate r depends on the condition number $\kappa(\hat{X})$: the more ill-conditioned the problem, the slower the linear rate.

Superlinear Convergence

For full BFGS (i.e. $\bar{m}$ large enough to store the entire history), [1, Theorem 8.6] guarantees superlinear convergence on strongly convex functions: the error ratio

$$\frac{\|w_{k+1} - w^*\|}{\|w_k - w^*\|} \longrightarrow 0. \tag{3.19}$$

Intuitively, as H_k accumulates more and more accurate curvature information, it converges to the true H^{-1}, and the search direction $p_k \approx -H^{-1}\nabla f_k$ becomes increasingly close to the Newton step.

Chapter 4

L-BFGS Experimental Results

All experiments use the matrix $X \in \mathbb{R}^{m \times n}$ with $m = 500$ samples and $n = 12$ features after standard preprocessing (zero mean, unit variance per feature). The response vector $y \in \mathbb{R}^n$ is drawn from a standard normal distribution with a fixed random seed (`seed` $= 42$). The reference solution w^* is computed via the normal equations $(XX^T + \lambda^2 I)\, w = Xy$ using `numpy.linalg.solve`, with a baseline solve time of approximately 0.3 s.

Unless stated otherwise, we use the default configuration obtained from a grid search over the algorithm's hyperparameters (with the initial point fixed to $w_0 = 0$, the canonical neutral choice): regularization $\lambda = 0.5$ (yielding $\kappa(\hat{X}) = 151.49$), memory size $\bar{m} = 10$, exact line search, the Barzilai–Borwein H_0 scaling (`bb1`), and the curvature-based restart enabled. The relative tolerance is $\varepsilon = 10^{-14}$ unless noted otherwise. Wall-clock measurements use `time.perf_counter()` and report the median over five repetitions after a discarded warm-up run; reported FLOP counts use the per-iteration operation model derived in Section 4.3.2.

The experimental discussion is organised in three parts. We first verify the *theoretical convergence properties* established in Section 3.0.6 (linear and superlinear regimes, Newton-direction alignment, and a comparison against representative first-order baselines). We then examine the *algorithmic design choices* of our L-BFGS variant (line search, memory size, H_0 scaling, restart strategy, Wolfe constants). Finally, we probe the *numerical and computational behaviour* of the method (sensitivity to initialisation, and the theoretical FLOP/storage cost together with its scalability in m).

4.1 Theoretical Convergence Properties

4.1.1 Empirical Linear and Superlinear Convergence Rates

We track the convergence behaviour at three memory sizes: $\bar{m} = 3$ (severely limited), $\bar{m} = 10$ (typical), and $\bar{m} = m = 500$ (effectively full BFGS, since the algorithm always converges in fewer than m iterations). We record two per-iteration quantities: the Q-linear ratio $r_k^{\text{lin}} = (f_k - f^*)/(f_{k-1} - f^*)$, whose stabilisation at a constant value below 1 is the empirical signature of the linear bound in Eq. (3.18), and the Q-superlinear ratio $r_k^{\text{sup}} = \|w_{k+1} - w^*\| \,/\, \|w_k - w^*\|$, whose decay towards 0 is the signature of the superlinear regime of Eq. (3.19). As in the default configuration, the curvature-based restart is enabled, and we run at the tight tolerance $\varepsilon = 10^{-14}$.

Table 4.1: L-BFGS at three memory sizes ($\lambda = 0.5$, exact LS, restart enabled, $\varepsilon = 10^{-14}$).

$\bar{m}$	**Iterations**	$\|w - w^*\|$	$f - f^*$
3	287	9.43×10^{-11}	2.78×10^{-17}
10	18	5.44×10^{-12}	2.78×10^{-17}
500	18	5.44×10^{-12}	2.78×10^{-17}

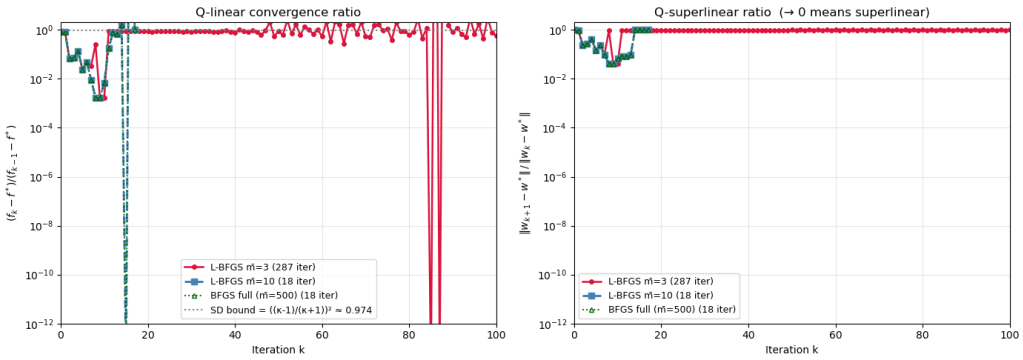

Figure 4.1: Q-linear (left) and Q-superlinear (right) convergence ratios

The two larger memories, $\bar{m} = 10$ and $\bar{m} = 500$, produce *identical* trajectories and converge in just 18 iterations. In the Q-superlinear panel (right) their ratio descends smoothly to about 10^{-1} before reaching the floor: the stored pairs build a stable, accurate approximation of the inverse Hessian, so every step is consistently effective—the behaviour expected of the superlinear regime of Eq. (3.19). The Q-linear panel (left) shows the characteristic "saw-tooth" of exact line search on a quadratic, alternating ordinary steps (ratio near 1) with exceptionally effective ones (the downward spikes), while always remaining below the steepest-descent bound $r_{\text{SD}} \approx 0.97$—confirming the linear bound of Eq. (3.18) and that L-BFGS converges far faster than any first-order method.

The severely limited memory $\bar{m} = 3$ tells a different story. It follows the same initial descent but cannot sustain it: with too few stored pairs, the curvature approximation repeatedly degrades, and the algorithm needs 287 iterations—over fifteen times more—to reach the same floor. The restart accelerates this case substantially: without it, $\bar{m} = 3$ still converges but takes 1,147 iterations, so the restart yields a roughly fourfold speed-up here (its effect on small memories is examined in detail in Section 4.2.6).

The contrast between $\bar{m} = 3$ and $\bar{m} \geq 10$ illustrates the key property of L-BFGS: a small memory matches the full-memory method *exactly*, provided it exceeds the *effective rank* of the problem. By Lemma 3, XX^T has only $n = 12$ non-zero eigenvalues, so only n Hessian directions carry non-trivial curvature; a memory of $\bar{m} \gtrsim n$ captures all of them and any further pairs are redundant. Storing 10 vectors is therefore as good as storing all 500—and dropping below the effective rank, as with $\bar{m} = 3$, degrades convergence sharply.

4.1.2 Comparison with Steepest Descent and Conjugate Gradient

To assess whether the curvature information used by L-BFGS provides a practical advantage, we compare it with two natural baselines: *steepest descent* (SD) with exact line search, and the linear *conjugate gradient* method (CG) applied to the Symmetric Positive Definite system $Hw = Xy$, where H-vector products are computed implicitly as $Hv = X(X^Tv) + \lambda^2 v$ at the same $O(mn)$ cost per

iteration as L-BFGS. All methods start from $w_0 = 0$ and use the relative stopping rule of Eq. (3.17) with $\varepsilon = 10^{-14}$. Two regimes are considered: well-conditioned ($\lambda = 1$, $\kappa \approx 76$) and ill-conditioned ($\lambda = 10^{-4}$, $\kappa \approx 7.6 \times 10^5$).

Table 4.2: Iteration counts and final gradient norms across four methods ($m = 500$, $n = 12$, exact line search, $\varepsilon = 10^{-14}$).

	$\lambda = 1$ ($\kappa \approx 76$)		$\lambda = 10^{-4}$ ($\kappa \approx 7.6 \times 10^5$)	
Method	iter	$\|\nabla f\|$	iter	$\|\nabla f\|$
Steepest Descent	1447	6.14×10^{-13}	1725	6.55×10^{-13}
Conjugate Gradient	15	5.47×10^{-15}	15	1.90×10^{-14}
L-BFGS ($\bar{m} = 10$)	19	1.47×10^{-10}	31	7.37×10^{-11}
Full BFGS ($\bar{m} = 500$)	19	1.47×10^{-10}	31	7.37×10^{-11}

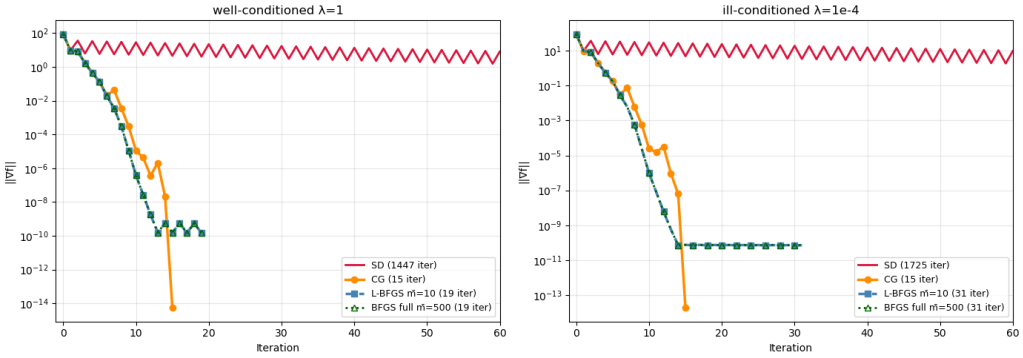

Figure 4.2: Convergence of the four methods ($\|\nabla f\|$ vs. iteration) in the well-conditioned ($\lambda = 1$, left) and ill-conditioned ($\lambda = 10^{-4}$, right) regimes. The x-axis is truncated at 60 iterations; SD continues well beyond this range (over 1,400 iterations in both cases).

The four methods separate into three performance tiers. Steepest descent is uncompetitive in both regimes, requiring over 1,400 iterations even on the well-conditioned problem: using only the gradient, it makes slow progress along the elongated level sets. L-BFGS with $\bar{m} = 10$ and full BFGS with $\bar{m} = 500$ produce *identical* trajectories (for the reason given in Section 4.1.1), cutting the iteration count by nearly two orders of magnitude relative to SD. Conjugate gradient, however, is faster still, reaching machine precision in just 15 iterations.

CG's strong performance is a direct consequence of the problem's algebraic structure. For SPD systems, CG terminates in a number of iterations equal to the number of *distinct* eigenvalues of H, not its dimension m. As noted in Section 4.1.1, XX^T has only $n = 12$ non-zero eigenvalues (Lemma 3); the spectrum of $H = XX^T + \lambda^2 I$ therefore consists of just $n + 1 = 13$ distinct values—the 12 shifted singular values plus λ^2 with multiplicity $m - n$. CG resolves this clustered spectrum in roughly n steps regardless of κ, which is why it takes 15 iterations in both regimes.

Two further effects deserve note. First, conditioning affects L-BFGS but not CG: raising κ from 76 to 7.6×10^5 increases the L-BFGS iteration count from 19 to 31, while CG stays at 15. CG's finite-termination property is immune to κ, whereas L-BFGS, as a general quasi-Newton method, is mildly sensitive to it—though far less so than SD, whose count also grows with κ (1447 $\rightarrow$ 1725). Second, CG reaches a higher final accuracy (10^{-14}–10^{-15} vs. 10^{-10}–10^{-11} for L-BFGS), a consequence of its exact-arithmetic optimality on quadratics.

The takeaway is not that L-BFGS is inferior, but that CG wins here precisely because our problem meets its two ideal assumptions at once: it is *quadratic* and has a *low-rank, clustered spectrum*. Neither holds in general—on non-quadratic objectives the conjugacy machinery breaks down, and on problems with non-clustered spectra CG loses its fast termination. L-BFGS degrades gracefully in both cases, making it the more robust choice for the broader class of problems where that structure is absent.

4.1.3 Alignment with the Newton Direction

While the previous experiments *observed* superlinear convergence, this one identifies its *cause*. As anticipated in Section 3.0.6, Nocedal and Wright [1, Thm. 8.6] attribute the superlinear rate of BFGS to the search direction $p_k = -H_k \nabla f_k$ becoming an increasingly accurate approximation of the Newton direction $p_k^{\mathrm{N}} = -H^{-1}\nabla f_k$. We test this directly by measuring the alignment defect $1 - \cos\theta_k$ between the two directions, using a cached Cholesky factorisation of H as a measurement oracle. Although this factorisation costs $O(m^3)$, it is used *only* to compute the reference Newton direction and is *not* part of the L-BFGS algorithm, which remains $O(mn + m\bar{m})$ per iteration. To keep the angle measurement clean, this instrumental variant runs without the curvature-based restart, so that each trajectory reflects a single, uninterrupted curvature approximation.

Table 4.3: Alignment defect $1 - \cos\theta_k$ at the last *clean* iteration (before $\|\nabla f\|$ reaches the noise floor 10^{-9}).

$\bar{m}$	**Clean iterations**	**Final** $1 - \cos\theta_k$
3	40	2.83×10^{-1}
10, 500	13	3.53×10^{-3}

The results confirm the mechanism. For $\bar{m} \geq 10$ the defect falls by two orders of magnitude, reaching $\sim 3.5 \times 10^{-3}$: the L-BFGS direction becomes nearly indistinguishable from the Newton step, which is exactly what drives the superlinear regime seen in Section 4.1.1. For $\bar{m} = 3$ the defect stalls at $\sim 2.8 \times 10^{-1}$ and never improves: with too few stored pairs, H_k cannot approximate H^{-1} closely enough to recover the Newton direction. This is the direct cause of its linear-only, much slower convergence—the 287 iterations of Section 4.1.1 are the price of a search direction that stays misaligned with Newton's. The series are truncated once $\|\nabla f\|$ falls below 10^{-9}, below which both directions are computed on numerical noise.

This experiment closes the theoretical loop of Section 3.0.6: the superlinear convergence of large-memory BFGS, the linear-only convergence of severely limited L-BFGS, and the iteration counts of Sections 4.1.1 and 4.1.2 are all manifestations of the same underlying mechanism—the quality of the curvature approximation built by the two-loop recursion.

4.2 Algorithmic Design Choices

Each experiment in this section isolates a single design decision in our L-BFGS implementation and quantifies its impact on convergence. Together, these experiments justify the default configuration used in the remainder of the chapter (`line_search='exact'`, $\bar{m} = 10$, `h0_scaling='bb1'`, `use_restart=True`, $\xi = 0.2$).

4.2.1 Exact Line Search vs. Strong Wolfe Line Search

We compare the two line-search strategies at a tight relative tolerance $\varepsilon = 10^{-14}$, using the standard Wolfe constants $(c_1, c_2) = (10^{-4}, 0.9)$ recommended by Nocedal and Wright [1, Sec. 3.1] for quasi-Newton methods.

Table 4.4: Comparison of line-search strategies ($m = 500$, $n = 12$, $\lambda = 0.5$, $\bar{m} = 10$, $\varepsilon = 10^{-14}$).

Line Search	**Iterations**	$\|w - w^*\|$	**Time (s)**	**Stop reason**
Exact	18	5.44×10^{-12}	0.001	stagnation
Wolfe ($c_1 = 10^{-4}, c_2 = 0.9$)	599	5.21×10^{-8}	0.023	stagnation

Exact LS achieves nearly four orders of magnitude higher accuracy (5.4×10^{-12} vs. 5.2×10^{-8}) in only 18 iterations, against the 599 that Wolfe needs before stalling. Its closed-form minimiser, Eq. (3.16), exploits the quadratic structure perfectly at the cost of a single matrix-vector product $X^T p_k \in O(mn)$. The Wolfe variant does not diverge but *stalls*: after roughly the first 100 iterations the step length stabilises around $\alpha \approx 0.07$, too large to trigger the stagnation safeguard but too small for the curvature condition to extract further progress. The cubic interpolation inside the bracketing-and-zoom loop loses accuracy once $\|\nabla f\| \lesssim 10^{-5}$, the working precision of the line search itself, and the run eventually stops on stagnation at 599 iterations.

An important caveat is that the Wolfe defaults $(c_1, c_2) = (10^{-4}, 0.9)$ are the standard recommendation for *non-quadratic* quasi-Newton problems. The next experiment shows that this default is inappropriate for our quadratic structure: with a stricter c_2, Wolfe LS becomes competitive again.

4.2.2 Sensitivity to the Wolfe Constants c_1, c_2

Building on the previous experiment, we test whether the poor performance of Wolfe LS is intrinsic or a side effect of the standard $c_2 = 0.9$. We sweep $c_1 \in \{10^{-4}, 10^{-3}, 10^{-2}\}$ and $c_2 \in \{0.1, \ldots, 0.9\}$, recording iterations and final accuracy.

Table 4.5: Effect of the Wolfe constants on convergence ($m = 500$, $n = 12$, $\lambda = 0.5$, $\bar{m} = 10$, $\varepsilon = 10^{-14}$). The three c_1 columns are identical, confirming that c_1 has no effect.

	Iterations (by c_1)			
c_2	10^{-4}	10^{-3}	10^{-2}	$\|w - w^*\|$
0.1	613	613	613	6.54×10^{-8}
0.3	26	26	26	1.27×10^{-9}
0.5	25	25	25	1.88×10^{-9}
0.7	31	31	31	2.11×10^{-9}
0.9	599	599	599	5.21×10^{-8}

Two observations stand out. First, c_1 has *no* effect: all three values produce identical iteration counts and accuracies. The Armijo condition is loose enough to be automatically satisfied at the curvature-determined trial step, so c_1 is structurally inactive here.

Second, and more interestingly, the dependence on c_2 is *non-monotone*: Wolfe LS performs well across a *broad* interior range and fails only at the two extremes. For $c_2 \in [0.2, 0.8]$ it converges in roughly 22–34 iterations—competitive with exact LS (18)—whereas both $c_2 = 0.1$ and $c_2 = 0.9$ collapse to

$\sim$ 600 iterations. The two failures have opposite causes. A large $c_2 = 0.9$ makes the curvature condition too permissive: it accepts steps that barely reduce the directional derivative, yielding poor curvature pairs and slow progress. A small $c_2 = 0.1$ makes it too restrictive: the line search struggles to locate a step tight enough to satisfy it, and the cubic interpolation stalls before doing so. Only an intermediate c_2 balances the two.

This experiment reframes the conclusion of Section 4.2.1: the apparent "Wolfe failure" was caused entirely by the default $c_2 = 0.9$ landing on a bad extreme. With an intermediate value such as $c_2 = 0.5$, Wolfe LS converges in 25 iterations—competitive with, though still slightly slower than, exact LS at 18 iterations.

4.2.3 Effect of the Memory Parameter $\bar{m}$

We vary $\bar{m} \in \{3, 5, 10, 20, 40\}$ using exact line search at the tight tolerance $\varepsilon = 10^{-14}$, with the curvature-based restart enabled (as in the default configuration).

Table 4.6: Effect of memory size $\bar{m}$ on L-BFGS convergence ($m = 500$, $n = 12$, $\lambda = 0.5$, exact LS, restart enabled, $\varepsilon = 10^{-14}$).

$\bar{m}$	**Iterations**	$\|w - w^*\|$
3	287	9.43×10^{-11}
5	605	2.76×10^{-10}
10	18	5.44×10^{-12}
20	18	5.44×10^{-12}
40	18	5.44×10^{-12}

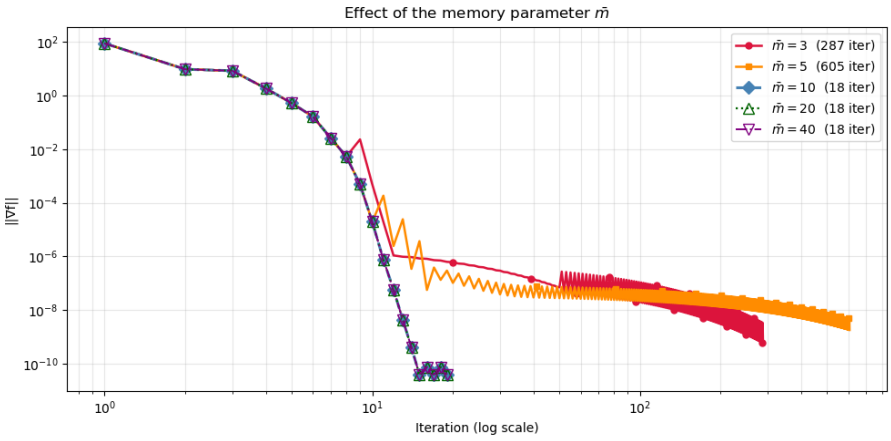

Figure 4.3: Gradient-norm convergence for $\bar{m} \in \{3, 5, 10, 20, 40\}$ (log–log scale, iteration counts in the legend). The curves for $\bar{m} \geq 10$ overlap and reach the floor in 18 iterations. Note the non-monotonicity: $\bar{m} = 5$ (orange) lies *above* $\bar{m} = 3$ (red), taking more than twice as many iterations despite using more memory.

From $\bar{m} = 10$ onwards the iteration count saturates at 18 and the accuracy plateaus at $\|w - w^*\| \approx 5.4 \times 10^{-12}$. This is the same rank-saturation effect established in Section 4.1.1: a memory of $\bar{m} \gtrsim n = 12$ captures all the non-trivial curvature directions, so $\bar{m} = 10$, 20, and 40 are indistinguishable. More striking is the *non-monotone* behaviour at small $\bar{m}$: $\bar{m} = 3$ takes 287 iterations, yet $\bar{m} = 5$—with *more* memory—takes 605, more than twice as many. This is the documented non-monotone depen-

dence of L-BFGS on memory size [1, Sec. 9.1]: for certain $\bar{m}$ below the effective rank $n = 12$, the stored curvature pairs interact unfavourably with the true Hessian spectrum, slowing convergence without affecting the final accuracy. Adding memory in this regime is no guarantee of faster convergence, since a larger but still-deficient set of pairs can distort the search direction more than a smaller one. The practical recommendation is therefore to set $\bar{m} \gtrsim n$, comfortably above this irregular regime.
The small-memory configurations are also where the curvature-based restart has its strongest effect. At the default $\bar{m} = 10$ the restart is dormant (it changes nothing), but at $\bar{m} = 3$ it speeds up convergence appreciably—from 1,147 to 287 iterations at $\lambda = 0.5$. Its effect in this regime is, however, not uniformly beneficial across conditioning; we return to this in Section 4.2.6.

4.2.4 Effect of the Regularization Parameter λ

We study convergence across five regularization regimes. Recall from Eq. (2.7) that $\kappa(\hat{X})$ scales as $O(1/\lambda)$ for small λ: weak regularization produces an ill-conditioned, elongated landscape, while large λ drives $\kappa(\hat{X}) \to 1$, making the problem nearly spherical.

Table 4.7: Effect of λ on condition number and convergence ($m = 500$, $n = 12$, exact LS, $\bar{m} = 10$, $\varepsilon = 10^{-14}$).

λ	$\kappa(\hat{X})$	**Iter**	$\|w - w^*\|$	t_{LBFGS} (ms)
10^{-4}	7.6×10^5	31	1.11×10^{-5}	1.23
10^{-2}	7.6×10^3	15	1.11×10^{-9}	0.69
1	75.8	19	1.81×10^{-11}	1.01
10^2	1.3	6	3.26×10^{-18}	0.59
10^4	1.0	3	3.99×10^{-22}	0.09

Three observations emerge. First, the iteration count stays small and only weakly dependent on conditioning: it ranges from 15 to 31 across the three ill- to moderately-conditioned regimes (κ from 76 to 7.6×10^5), without growing monotonically with κ. This confirms that L-BFGS with exact LS converges in $O(n)$ iterations largely independent of κ, consistent with the conjugate-direction interpretation of memoryless BFGS [1, Sec. 9.1] and with the rank-12 structure of XX^T (Lemma 3). Second, although the iteration count stays low, *final accuracy degrades sharply with* κ: from 4×10^{-22} at $\lambda = 10^4$ to 1.1×10^{-5} at $\lambda = 10^{-4}$. This is not a failure of the algorithm but the intrinsic floating-point floor of the normal-equations formulation: forming XX^T squares the condition number (Eq. (2.6)), so the attainable error is bounded below by roughly $\kappa^2 \cdot \varepsilon_{\text{mach}}$. At $\lambda = 10^{-4}$ this gives $\sim 10^{-4}$, consistent with the observed 1.1×10^{-5}.
Third, in the regime $\lambda \gg \sigma_{\max}(X)$ the regularisation term dominates, $H \approx \lambda^2 I$, and L-BFGS reduces to scaled gradient descent on a near-spherical problem, converging in 3–6 iterations. Across all regimes L-BFGS solves the problem in about 1 ms. Section 4.3.2 examines how this cost scales with m.
Finally, this experiment clarifies why we adopt $\lambda = 0.5$ as the default for the rest of the chapter. The parameter λ is not an algorithmic knob but part of the problem definition: it sets the regularisation strength, and hence the trade-off between fitting the data and controlling $\|w\|$. Although large λ makes the problem numerically trivial (the case $\lambda = 10^4$ converges in 3 iterations to machine precision), the resulting solution is dominated by the penalty term $\lambda^2 \|w\|^2$ and is essentially $w \approx 0$, ignoring the data entirely. The value $\lambda = 0.5$ instead yields a moderately ill-conditioned instance ($\kappa \approx 151$) that is representative of realistic regularisation and exercises the algorithm in its non-trivial regime—neither

so well-conditioned that convergence is immediate, nor so ill-conditioned that the problem becomes numerically unsolvable.

4.2.5 H_0 Scaling Strategies

We compare the three strategies introduced in Section 3.0.2 for the initial Hessian approximation $H_k^0 = \gamma_k I$ used in the two-loop recursion: the Nocedal default (BB2, Eq. (3.6)), the Barzilai–Borwein companion (BB1, Eq. (3.7)), and the safeguarded variant. To show that the comparison does not depend on the restart, we run each strategy both with and without it, in both conditioning regimes.

Table 4.8: Effect of H_0 scaling on L-BFGS convergence, with and without the curvature-based restart ($m = 500$, $n = 12$, exact LS, $\bar{m} = 10$, $\varepsilon = 10^{-14}$).

		Well-cond. ($\lambda = 1$, $\kappa \approx 76$)		*Ill-cond.* ($\lambda = 10^{-2}$, $\kappa \approx 7.6 \times 10^3$)	
Strategy	**Restart**	iter	$\|w - w^*\|$	iter	$\|w - w^*\|$
`nocedal`	off	38	3.82×10^{-12}	29	1.11×10^{-9}
`nocedal`	on	38	3.82×10^{-12}	29	1.11×10^{-9}
`bb1`	off	19	1.81×10^{-11}	15	1.11×10^{-9}
`bb1`	on	19	1.81×10^{-11}	15	1.11×10^{-9}
`safeguarded`	off	38	3.82×10^{-12}	29	1.11×10^{-9}
`safeguarded`	on	38	3.82×10^{-12}	29	1.11×10^{-9}

Two clear results emerge. First, `bb1` converges in roughly *half* the iterations of the other two strategies—19 vs. 38 in the well-conditioned regime, 15 vs. 29 in the ill-conditioned one—and this gap is *identical with and without the restart*, confirming that it is a genuine property of the scaling rule rather than an interaction with the restart. The two BB formulas are related by the Cauchy–Schwarz inequality, which gives $\gamma^{\mathrm{BB1}} \geq \gamma^{\mathrm{N}}$: the larger initial scale of `bb1` produces more effective early steps on this problem, at the cost of a marginally higher final residual (1.8×10^{-11} vs. 3.8×10^{-12}, both at the floor). This advantage is exactly what led the grid search to select `bb1` as the default.
Second, `nocedal` and `safeguarded` produce *identical* results to the last digit. The safeguard clips γ into a safe interval to prevent it from exploding near the solution (where $y^T y \to 0$), but on this well-behaved quadratic γ never approaches the clipping bounds, so the safeguard never activates. It is thus free insurance—inert here, but a cheap guarantee of robustness on problems where γ might otherwise degenerate.

4.2.6 Curvature-Based Restart

The curvature-based restart of Liu and Nocedal [3, Sec. 4] clears the L-BFGS memory whenever the curvature ratio $\gamma_k = (s^T y)/(y^T y)$ drops by more than a factor ξ relative to the previous iteration (Eq. (3.10)), signalling entry into a region of qualitatively different curvature. We test it against a no-restart control at two memory sizes: the default $\bar{m} = 10$ and the severely limited $\bar{m} = 3$, in both conditioning regimes.

Table 4.9: Effect of the curvature-based restart ($\xi = 0.20$) at two memory sizes ($m = 500$, $n = 12$, exact LS, `bb1`, $\varepsilon = 10^{-14}$).

		Well-cond. ($\lambda = 1$)		*Ill-cond.* ($\lambda = 10^{-2}$)	
$\bar{m}$	**Restart**	iter	n_memory_clears	iter	n_memory_clears
10	off	19	1	15	1
10	$\xi = 0.20$	19	1	15	1
3	off	897	1	332	1
3	$\xi = 0.20$	399	3	848	3

The two memory sizes tell opposite stories. With the default $\bar{m} = 10$ the restart is *dormant*: enabling it leaves the iteration count unchanged in both regimes (19 and 15). The single memory clear counted in these runs is not the ξ-test at all, but the negative-curvature reset triggered when $y^T s$ becomes negligibly small near the floor (cf. Section 3.0.2). A sweep over $\xi \in \{0.05, 0.10, 0.20, 0.40, 0.70\}$ confirms this: all five thresholds give identical results (15 iterations), so the ξ-test never fires when the memory is adequate.

The reason is structural. The objective is quadratic, so H is constant and γ_k (Eq. (3.9)) stays bounded between $1/\lambda_{\max}(H)$ and $1/\lambda_{\min}(H)$. With $\bar{m} \geq n$ the curvature approximation is stable throughout, so γ_k never drops by the factor required to trigger a reset. The mechanism is therefore inert at the default memory size, adding only negligible overhead.

With the starved $\bar{m} = 3$, by contrast, the ξ-test *does* fire (three resets in each run), and its effect is substantial but *not uniformly beneficial*. In the well-conditioned regime it more than halves the iteration count ($897 \to 399$), as clearing the stale, rank-deficient memory lets the method rebuild a fresher curvature estimate. In the ill-conditioned regime, however, it makes matters *worse* ($332 \to 848$): here the discarded pairs, though imperfect, still carried useful low-curvature information, and rebuilding from scratch repeatedly costs more than it saves. The restart thus trades stale information for freshness—a trade that pays off only when the stale information is genuinely harmful.

For our purposes the conclusion is clear. At the default $\bar{m} = 10$ the restart is inactive and costless, so we leave it enabled as a no-risk safeguard: it never interferes when the memory is adequate, and remains available should a smaller memory be used. Its variable behaviour at $\bar{m} = 3$ is a further argument for choosing $\bar{m} \gtrsim n$, where the question does not arise.

4.3 Numerical and Computational Scalability

The previous sections established the theoretical and algorithmic behaviour of L-BFGS on the default ML-CUP instance. We now probe two remaining practical aspects: how robust the method is to the choice of starting point, what its theoretical computational cost is, and how it scales empirically with the problem dimensions n and m.

4.3.1 Sensitivity to the Initial Iterate w_0

As established in Section 3.0.6, L-BFGS converges from any starting point on a strongly convex quadratic, with a rate depending only on κ. We test four starting points spanning four orders of magnitude in $\|w_0 - w^*\|$, from $w_0 = 0$ to a point at distance $\sim$2,000, plus one deliberately placed very close to the optimum.

Table 4.10: Sensitivity to the initial iterate ($m = 500$, $n = 12$, $\lambda = 0.5$, $\bar{m} = 10$, exact LS, $\varepsilon = 10^{-14}$).

Initialization	$\|w_0 - w^*\|$	**Iter**	$\|w - w^*\|$
`zeros`	0.85	18	5.44×10^{-12}
`random N(0, 1)`	22.0	18	1.91×10^{-12}
`random N(0, 100²)`	2.19×10^3	15	1.13×10^{-12}
`near optimum` ($w^* + 0.01 \cdot \mathcal{N}(0,1)$)	0.23	41	2.83×10^{-12}

The main result is that the *distance* to the optimum does not determine the iteration count. The three generic starts converge in 15–18 iterations even though $\|w_0 - w^*\|$ ranges over four orders of magnitude; remarkably, the *farthest* start (`N(0, 100²)`, at distance ∼2,200) converges in the *fewest* iterations (15). This is the global linear convergence guarantee of Eq. (3.18) in action: on a strongly convex quadratic the rate depends only on κ, not on w_0. The slight edge of the far start is a minor geometric effect—a generic point off the origin produces richer initial curvature pairs than the structured choice $w_0 = 0$—and it is precisely this artefact that led the grid search to rank random initialisations marginally above `zeros`; we nonetheless fix $w_0 = 0$ as the canonical neutral default.

The `near optimum` start is the apparent exception: despite being the *closest* ($\|w_0 - w^*\| = 0.23$), it takes the *most* iterations (41). This does not contradict the global convergence result; it is an artefact of the *relative* stopping criterion (Eq. (3.17)). Starting near w^* gives a small initial gradient $\|\nabla f_0\|$, so the absolute threshold $\varepsilon \cdot \|\nabla f_0\|$ becomes proportionally tighter, and more iterations are needed to squeeze out the extra orders of magnitude. The difficulty lies in the convergence *measure*, not in the problem itself: the final accuracy (2.8×10^{-12}) is the same as the other starts.

4.3.2 Theoretical Cost, Storage, and Scalability

The main practical motivation for L-BFGS is its favourable cost compared with a direct solver. Forming and factorising the normal equations $(XX^T + \lambda^2 I)\, w = Xy$ with `numpy.linalg.solve` costs $O(m^3)$, whereas L-BFGS costs $O\big((mn + m\bar{m})\, n_{\text{iter}}\big)$, with n_{iter} governed by the conditioning and effective rank rather than by m. We make this concrete with a FLOP count derived from the structure of the two-loop recursion (Algorithm 1). Per iteration,

$$\text{FLOPs/iter} = \underbrace{(4\bar{m} + 1)\, m}_{\text{two-loop}} + \underbrace{2mn}_{\text{gradient}} + \underbrace{mn + 2m}_{\text{exact LS}} + \underbrace{2m}_{\text{update}},$$

which is linear in both m and $\bar{m}$, while storage is $2\bar{m}$ vectors of length m plus $O(m)$ working space. Crucially, the *total* cost to convergence depends on both this per-iteration work *and* the iteration count, which combine non-trivially.

Table 4.11: Total FLOPs to convergence vs. $\bar{m}$ ($m = 500$, $n = 12$, $\lambda = 0.5$, exact LS, $\varepsilon = 10^{-14}$). Iteration counts are those of Table 4.6.

$\bar{m}$	**Iter**	**FLOPs/iter**	**Total FLOPs**	**Storage (MB)**
3	287	26,500	7,605,500	0.040
5	605	30,500	18,452,500	0.056
10	18	40,500	729,000	0.096
20	18	60,500	1,089,000	0.176
40	18	100,500	1,809,000	0.336

The total cost is strongly *non-monotone* in $\bar{m}$, driven entirely by the iteration count of Section 4.2.3. The configuration $\bar{m} = 5$ requires 18.5M FLOPs—over 25× more than the optimal $\bar{m} = 10$ at 0.73M—and even $\bar{m} = 3$ (7.6M) outperforms it, a direct FLOP-level manifestation of the rank-deficient Hessian structure. The sweet spot is $\bar{m} = 10$: just above the effective rank, it captures the relevant curvature with minimal redundancy. Increasing $\bar{m}$ further only inflates the per-iteration cost without reducing the iteration count, so the total grows linearly beyond this point. Storage is never a constraint: even $\bar{m} = 40$ uses only 0.34 MB, so $\bar{m}$ should be chosen for convergence speed alone.

Returning to the comparison with the direct solver, the FLOP model makes the asymptotic advantage explicit. At the optimal $\bar{m} = 10$, L-BFGS needs about 0.73M FLOPs on the present ML-CUP instance ($m = 500$). More generally, if the iteration count remains roughly constant, the total cost scales approximately linearly in m, since each iteration costs $O(mn + m\bar{m})$. The direct solver, by contrast, scales as $O(m^3)$. For the ML-CUP dimensions this gap is already substantial, and it widens by a further factor of about m^2 as the problem size grows, which is precisely the regime where the limited-memory approach becomes indispensable. We verify this scaling empirically—and confirm the governing role of the effective rank n—in Section 4.3.3.

4.3.3 Empirical Scaling with the Dimensions n and m

The cost analysis above, like all preceding experiments, is based on the fixed ML-CUP matrix ($m = 500$, $n = 12$). To verify directly how the method scales with the problem dimensions—and, in particular, to confirm that the iteration count is governed by the effective rank n rather than by the sample size m—we run two sweeps on synthetic Gaussian data with standardised columns. We report only iteration counts (which are deterministic and noise-free) together with $\kappa(\hat{X})$, computed exactly from the singular values of X; this separates the effect of *dimension* from that of *conditioning*.

Table 4.12: Scaling with the number of features n (fixed $m = 500$, exact LS, `bb1`, $\varepsilon = 10^{-14}$). The saturation memory $\bar{m}^*$ is the smallest $\bar{m}$ reaching the iteration plateau.

n	$\kappa(\hat{X})$	**iter L-BFGS**	**iter CG**	$\bar{m}^*$
5	1.17	5	5	3
12	1.28	18	12	10
25	1.50	19	18	23
50	1.84	33	25	48
100	2.46	48	36	98

Table 4.13: Scaling with the sample size m (fixed $n = 12$, exact LS, `bb1`, $\varepsilon = 10^{-14}$).

m	$\kappa(\hat{X})$	**iter L-BFGS**	**iter CG**
200	1.45	21	12
500	1.28	18	12
2000	1.15	14	11
8000	1.06	10	9

The two sweeps confirm the central structural claim of this chapter: the convergence of both methods is governed by the effective rank n, not by the ambient sample size m. As n grows (Table 4.12), at nearly constant conditioning, all three indicators scale with n. Conjugate gradient terminates in

almost exactly n iterations—$5, 12, 18, 25, 36$ for $n = 5, \ldots, 100$—the direct manifestation of its finite-termination property on the $n + 1$ distinct eigenvalues of H (Section 4.1.2). The L-BFGS iteration count grows in step, and, most tellingly, the saturation memory $\bar{m}^*$ tracks n closely $(3, 10, 23, 48, 98)$: the empirical rule $\bar{m} \gtrsim n$ established at $n = 12$ in Section 4.1.1 holds across the whole range, with the threshold shifting in lockstep with the effective rank.

Varying m at fixed n (Table 4.13) tells the complementary story: the iteration count does *not* grow with m. It in fact decreases slightly ($21 \to 10$ for L-BFGS), but this is a conditioning effect, not a dimension effect—larger Gaussian matrices have better-spread singular values, so κ falls from 1.45 to 1.06 over the range, and fewer iterations are needed. CG likewise stays essentially flat at $\sim n$. This is exactly the behaviour predicted by the FLOP model of Section 4.3.2: with the iteration count independent of m and a per-iteration cost of $O(mn + m\bar{m})$, the total work grows only linearly in m, against the $O(m^3)$ of the direct solver.

4.4 Summary

Our experimental results validate the theoretical predictions of Section 3.0.6 and provide several practical insights into the design of L-BFGS for this problem class.

On the *theoretical* side, the empirical convergence ratios confirm both the global linear bound of Liu and Nocedal [3, Thm. 7.1] and the superlinear convergence of full BFGS [1, Thm. 8.6], while the alignment between the L-BFGS and Newton directions provides a direct mechanistic explanation of the latter. The comparison against steepest descent and conjugate gradient positions L-BFGS correctly within the landscape of iterative methods: although CG outperforms L-BFGS on this specific problem (15 vs. 19 iterations) thanks to its finite-termination property on clustered spectra, L-BFGS would be the method of choice in non-quadratic settings or on problems with denser Hessian spectra.

On the *algorithmic* side, exact line search dominates the strong Wolfe variant for this quadratic problem, converging in 18 iterations against the 599 of the standard Wolfe defaults—though a less extreme curvature constant ($c_2 = 0.5$ rather than 0.9) largely closes the gap. The choice of H_0 scaling also matters: the `bb1` rule converges in roughly half the iterations of the Nocedal and safeguarded variants, which is why we adopt it as the default. The most striking finding is the *non-monotone* dependence on the memory size $\bar{m}$: a larger memory need not help, with $\bar{m} = 5$ (605 iterations) performing worse than $\bar{m} = 3$ (287), while any $\bar{m} \gtrsim n = 12$ saturates at 18. This indicates that the optimal memory size sits just above the effective rank of XX^T. The curvature-based restart is dormant at this default memory size, but becomes active—with a problem-dependent, not uniformly beneficial, effect—when the memory is starved below the effective rank.

On the *numerical* side, the per-iteration FLOP model confirms the favourable cost of L-BFGS: each iteration scales linearly in m, against the $O(m^3)$ of the direct solver, while the iteration count is governed by the conditioning and effective rank rather than by m. This picture is confirmed by varying the dimensions directly: the iteration count and the saturation memory $\bar{m}^*$ both track the effective rank n, while holding n fixed and increasing m leaves the iteration count essentially unchanged. The total cost is itself non-monotone in $\bar{m}$—mirroring the iteration count—with the optimal $\bar{m} = 10$ requiring some $25\times$ fewer FLOPs than $\bar{m} = 5$. Finally, the attainable accuracy tracks the conditioning of the problem: as κ ranges from 1 to $\sim 10^6$, the final error degrades from $\sim 10^{-22}$ to $\sim 10^{-5}$, consistent with the floating-point floor $\kappa^2 \varepsilon_{\text{mach}}$ of the normal-equations formulation.

Chapter 5

QR Factorization

5.1 QR Factorization

We now turn from the optimization point of view to the direct linear algebra point of view. The same regularized least squares problem can be solved without generating an iterative sequence of approximations: one factorizes the augmented matrix

$$A \coloneqq \hat{X} = \begin{bmatrix} X^T \\ \lambda I_m \end{bmatrix} \in \mathbb{R}^{(n+m)\times m}, \qquad b \coloneqq \hat{y} = \begin{bmatrix} y \\ 0 \end{bmatrix} \in \mathbb{R}^{n+m}, \tag{5.1}$$

and then solves the resulting triangular problem. Since $\lambda > 0$, Lemma 1 shows that A has full column rank. Therefore the least squares problem

$$\min_{w\in\mathbb{R}^m} \|Aw - b\| \tag{5.2}$$

has a unique minimizer.

5.1.1 Thin QR Factorization

For a full column rank matrix $A \in \mathbb{R}^{p\times m}$, with $p = n + m \geq m$, the thin QR factorization is

$$A = Q_1 R, \qquad Q_1 \in \mathbb{R}^{p\times m}, \quad Q_1^T Q_1 = I_m, \quad R \in \mathbb{R}^{m\times m}, \tag{5.3}$$

where R is upper triangular and nonsingular.

Let $Q_2 \in \mathbb{R}^{p\times n}$ be a matrix whose columns form an orthonormal basis of the orthogonal complement of $\mathrm{col}(Q_1)$. Then

$$Q = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \in \mathbb{R}^{p\times p} \tag{5.4}$$

is an orthogonal matrix:

$$Q^T Q = Q Q^T = I_p. \tag{5.5}$$

Therefore,

$$Q_1^T Q_1 = I_m, \qquad Q_2^T Q_2 = I_n, \qquad Q_2^T Q_1 = 0. \tag{5.6}$$

Hence we have that

$$Q^T A = Q^T Q_1 R = \begin{bmatrix} R \\ 0 \end{bmatrix}, \qquad Q^T b = \begin{bmatrix} Q_1^T b \\ Q_2^T b \end{bmatrix} = \begin{bmatrix} c \\ d \end{bmatrix}, \quad c \in \mathbb{R}^m,\ d \in \mathbb{R}^n. \tag{5.7}$$

The orthogonal transformations preserve the Euclidean norm, hence

$$\|Aw - b\|^2 = \left\|Q^T(Aw - b)\right\|^2 = \|Rw - c\|^2 + \|d\|^2 . \tag{5.8}$$

Observing that he term $\|d\|^2$ is independent of w, we can rewrite the minimization problem like follows

$$\arg\min_{w \in \mathbb{R}^m} \|Aw - b\|^2 = \arg\min_{w \in \mathbb{R}^m} \|Rw - c\|^2 . \tag{5.9}$$

Since R is nonsingular, there exists a unique w such that

$$Rw - c = 0, \qquad \text{that is,} \qquad Rw = c = Q_1^T b. \tag{5.10}$$

Since $A^T A = (Q_1 R)^T (Q_1 R)$, this is the QR analogue of solving the normal equations. Indeed,

$$R^T R = A^T A = XX^T + \lambda^2 I_m, \qquad R^T c = A^T b = Xy. \tag{5.11}$$

Thus QR and the normal equations define the same exact solution. Numerically, however, QR is preferable because it avoids explicitly forming and solving with $A^T A$, whose condition number is squared:

$$\kappa(A^T A) = \kappa(A)^2. \tag{5.12}$$

For small λ, this distinction is important, since Eq. (2.7) shows that $\kappa(A)$ already grows like $O(1/\lambda)$.

5.1.2 Algorithmic Structure

The QR-based solver can be summarized as follows:

Algorithm 3 QR Solver for the Regularized Least Squares Problem

Require: Data matrix $X \in \mathbb{R}^{m \times n}$; response $y \in \mathbb{R}^n$; parameter $\lambda > 0$

1: Form the linear operator $A = [X^T; \lambda I_m]$ and vector $b = [y; 0]$
2: Compute a thin QR factorization $A = Q_1 R$
3: Compute $c \leftarrow Q_1^T b$
4: Solve $Rw = c$ by back-substitution
5: **return** w

In practice, the matrix Q_1 is not formed explicitly. Forming it would require $O((n+m)m)$ storage and applying it as a dense matrix would obscure the sparse and structured nature of A. Instead, the orthogonal transformations used during the factorization are stored in compact form and later applied to b.

5.1.3 Cost for the QR Solver

For a generic dense matrix $A \in \mathbb{R}^{p\times m}$, with $p = n + m \geq m$, computing a thin QR factorization requires

$$O(pm^2) = O((n+m)m^2) \tag{5.13}$$

operations. Once the factorization is available, computing the transformed right-hand side $c = Q_1^T b$ requires at most $O(pm)$ additional operations, while solving the upper triangular system

$$Rw = c \tag{5.14}$$

by back-substitution costs $O(m^2)$. Therefore, the factorization dominates the overall computational cost. Since $p = n + m$, treating the augmented matrix A as a generic dense matrix gives

$$O((n+m)m^2) = O(nm^2 + m^3), \tag{5.15}$$

which contains a cubic term in m. This is the baseline complexity of the QR solver before exploiting the particular block structure of the augmented matrix. The structured implementation developed in Section 5.2 reduces this cost substantially.

5.2 QR with Householder Reflectors

Householder reflectors are the standard stable mechanism for computing QR factorizations [4, 5]. A Householder reflector is an orthogonal matrix of the form

$$H = I - \tau u u^T, \qquad \tau = \frac{2}{u^T u}, \tag{5.16}$$

where $u \neq 0$. It is symmetric and orthogonal:

$$H^T = H, \qquad H^T H = I. \tag{5.17}$$

Given a vector $z \in \mathbb{R}^\ell$, the reflector is chosen so that all components below the first one are annihilated. Let

$$\alpha = -\operatorname{sign}(z_1) \|z\|_2, \qquad u = z - \alpha e_1. \tag{5.18}$$

Then

$$Hz = \alpha e_1 = \begin{bmatrix} -\operatorname{sign}(z_1)\|z\|_2 \\ 0 \\ \vdots \\ 0 \end{bmatrix}. \tag{5.19}$$

Here sign(0) is taken as 1. The sign choice avoids cancellation when z_1 is already close to $\|z\|_2$, and is therefore important for numerical stability.

Thus, at step i, the Householder reflector H is chosen such that the current vector z_i is mapped to one whose entries below position i are zero, while the i-th entry becomes α.

5.2.1 Dense Householder QR

For a generic dense matrix $A \in \mathbb{R}^{p\times m}$, Householder QR proceeds column by column. At step k, one constructs a reflector H_k acting on matrix A only on the elements of rows $k, \ldots, p$ of the k-th column, such that

$$H_k A_{k:p,k} = \begin{bmatrix} \alpha_k \\ 0 \\ \vdots \\ 0 \end{bmatrix}. \tag{5.20}$$

The same reflector is applied to the whole trailing submatrix $A_{k:p,k:m}$. After m steps we get:

$$H_m \cdots H_2 H_1 A = \begin{bmatrix} R \\ 0 \end{bmatrix}. \tag{5.21}$$

Since each Householder reflector is orthogonal, their product is also orthogonal. Defining

$$Q^T = H_m \cdots H_2 H_1, \qquad Q \in \mathbb{R}^{p\times p}, \tag{5.22}$$

we obtain

$$Q^T A = \begin{bmatrix} R \\ 0 \end{bmatrix}, \quad R \in \mathbb{R}^{m\times m}, \tag{5.23}$$

where R is upper triangular. Multiplying by Q gives

$$A = Q \begin{bmatrix} R \\ 0 \end{bmatrix}. \tag{5.24}$$

Let $Q_1 \in \mathbb{R}^{p\times m}$ denote the first m columns of Q. Since the remaining rows of the block matrix above are zero, this reduces to the thin QR factorization

$$A = Q_1 R, \qquad Q_1^T Q_1 = I_m. \tag{5.25}$$

The important implementation point is that Q is never formed explicitly. At each iteration of the factorization, the same Householder reflector applied to A is also applied to the corresponding part of the right-hand side b:

$$b_{k:p} \leftarrow H_k b_{k:p} = b_{k:p} - \tau_k u_k \left(u_k^T b_{k:p}\right). \tag{5.26}$$

After all m reflectors have been applied, b contains $Q^T b$. Its first m entries are

$$c = Q_1^T b, \tag{5.27}$$

which is the right-hand side of the triangular system $Rw = c$.

Algorithm 4 Compact Dense Householder QR

Require: Matrix $A \in \mathbb{R}^{p\times m}$ with $p \geq m$

1: **for** $k = 1, \ldots, m$ **do**
2: $\quad z \leftarrow A_{k:p,k}$
3: $\quad \alpha \leftarrow -\text{sign}(z_1)\,\|z\|_2$
4: $\quad u \leftarrow z - \alpha e_1; \quad \tau \leftarrow 2/(u^T u)$
5: $\quad A_{k:p,k:m} \leftarrow A_{k:p,k:m} - \tau u(u^T A_{k:p,k:m})$
6: $\quad$ Store (u, τ)
7: **end for**
8: $R \leftarrow \text{triu}(A_{1:m,1:m})$
9: **return** R and stored reflectors

Algorithm 4 is the mathematically canonical version. It is useful for explaining QR, but for the project matrix it should be specialized as described next.

5.2.2 Structured Householder Row Insertion

The augmented matrix has the special block structure

$$A = \begin{bmatrix} X^T \\ \lambda I_m \end{bmatrix}, \qquad b = \begin{bmatrix} y \\ 0 \end{bmatrix}. \tag{5.28}$$

The order of the rows is irrelevant for the least-squares problem. Indeed, if P is a permutation matrix, then P is orthogonal and therefore

$$\|Aw - b\| = \|P(Aw - b)\|. \tag{5.29}$$

We can consequently work with the equivalent permuted system

$$\tilde{A} = \begin{bmatrix} \lambda I_m \\ X^T \end{bmatrix}, \qquad \tilde{b} = \begin{bmatrix} 0 \\ y \end{bmatrix}. \tag{5.30}$$

This ordering is particularly convenient because the first m rows already form an upper triangular matrix,

$$R_0 = \lambda I_m. \tag{5.31}$$

The remaining n rows of X^T can therefore be incorporated one at a time. At iteration i, let $x_i^T = [a_1, \ldots, a_m]$ be the new data row and y_i its corresponding right-hand-side entry. We maintain the current upper triangular factor R_{i-1} and the transformed right-hand side c_{i-1}.

Appending the new row gives

$$\begin{bmatrix} R_{i-1} \\ x_i^T \end{bmatrix} = \begin{bmatrix} R_{11} & R_{12} & R_{13} & \cdots & R_{1m} \\ 0 & R_{22} & R_{23} & \cdots & R_{2m} \\ 0 & 0 & R_{33} & \cdots & R_{3m} \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & R_{mm} \\ a_1 & a_2 & a_3 & \cdots & a_m \end{bmatrix} \in \mathbb{R}^{(m+1)\times m}. \tag{5.32}$$

A sequence of m Householder reflectors is then applied to eliminate the entries of the inserted row one at a time, yielding

$$\begin{bmatrix} R_{i-1} \\ x_i^T \end{bmatrix} \longrightarrow \begin{bmatrix} R_i \\ 0 \end{bmatrix} = \begin{bmatrix} R_{11}^{(i)} & R_{12}^{(i)} & R_{13}^{(i)} & \cdots & R_{1m}^{(i)} \\ 0 & R_{22}^{(i)} & R_{23}^{(i)} & \cdots & R_{2m}^{(i)} \\ 0 & 0 & R_{33}^{(i)} & \cdots & R_{3m}^{(i)} \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & R_{mm}^{(i)} \\ 0 & 0 & \cdots & 0 & 0 \end{bmatrix}. \tag{5.33}$$

The same orthogonal transformations must be applied to the corresponding right-hand side:

$$\begin{bmatrix} c_{i-1} \\ y_i \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_m \\ y_i \end{bmatrix} \longrightarrow \begin{bmatrix} c_1^{(i)} \\ c_2^{(i)} \\ \vdots \\ c_m^{(i)} \\ \eta_i \end{bmatrix} = \begin{bmatrix} c_i \\ \eta_i \end{bmatrix}. \tag{5.34}$$

Unlike the inserted row of the matrix, the last component η_i of the transformed right-hand side is not generally zero. Since it corresponds to a zero row in the transformed matrix, it is independent of w and therefore does not affect the least-squares minimizer. Consequently, only the updated vector c_i needs to be retained for the subsequent row insertions and for the final triangular solve $R_n w = c_n$. The scalar η_i may be discarded unless the final residual norm is also required.

At step k, the current matrix R is already upper triangular and the previous components $a_1, \ldots, a_{k-1}$ of the inserted row have already been annihilated. Therefore, in column k, the only nonzero entry below the diagonal is the active element a_k.

For this reason, it is not necessary to build a Householder reflector on the entire column. It is sufficient to consider only the two entries involved in the elimination,

$$z = \begin{bmatrix} R_{kk} \\ a_k \end{bmatrix} \in \mathbb{R}^2. \tag{5.35}$$

A two-dimensional Householder reflector is then constructed as

$$H_k^{(i)} = I_2 - \tau u u^T, \tag{5.36}$$

and chosen so that

$$H_k^{(i)} \begin{bmatrix} R_{kk} \\ a_k \end{bmatrix} = \begin{bmatrix} \rho \\ 0 \end{bmatrix}. \tag{5.37}$$

Thus, at each step only the pair (R_{kk}, a_k) is needed to determine the reflector: the diagonal entry is updated to ρ, while the corresponding entry of the inserted row is annihilated.

The same 2×2 reflector must then be applied to the remaining entries of the two affected rows, so that the transformation is consistent across the whole trailing part of the matrix:

$$\begin{bmatrix} R_{k,k:m} \\ a_{k:m} \end{bmatrix} \leftarrow H_k^{(i)} \begin{bmatrix} R_{k,k:m} \\ a_{k:m} \end{bmatrix}. \tag{5.38}$$

The corresponding right-hand-side entries are transformed by the same reflector,

$$\begin{bmatrix} c_k \\ \eta \end{bmatrix} \leftarrow H_k^{(i)} \begin{bmatrix} c_k \\ \eta \end{bmatrix}, \tag{5.39}$$

where η is initialized as $\eta = y_i$ at the beginning of the insertion of row x_i^T.

After repeating this procedure for $k = 1, \ldots, m$, all entries of the inserted row have been annihilated. The row can then be discarded, while the updated R and c contain the effect of the newly inserted data row.

Algorithm 5 Structured Householder QR for $[X^T; \lambda I_m]$

Require: $X \in \mathbb{R}^{m \times n}$, $y \in \mathbb{R}^n$, $\lambda > 0$

$R \leftarrow \lambda I_m$; $\quad c \leftarrow 0 \in \mathbb{R}^m$
for $i = 1, \ldots, n$ **do**
  $a \leftarrow X_{:,i}$; $\quad \eta \leftarrow y_i$
  for $k = 1, \ldots, m$ **do**
    $z \leftarrow [R_{kk},\, a_k]^T$
    Build the Householder reflector H such that $Hz = [\rho,\, 0]^T$
    $[R_{k,k:m};\, a_{k:m}] \leftarrow H[R_{k,k:m};\, a_{k:m}]$
    $[c_k;\, \eta] \leftarrow H[c_k;\, \eta]$
    Store the reflector parameters if later products with Q or Q^T are required
  end for
end for
Solve $Rw = c$ by back-substitution
return w

5.2.3 Complexity and Numerical Properties

The structured row-insertion procedure exploits two properties of the augmented matrix: the regularization block λI_m is already triangular, and only the n dense rows of X^T need to be incorporated. For each inserted row, the algorithm performs m two-dimensional Householder transformations. At iteration k, the reflector is applied only to the trailing row segments

$$R_{k,k:m} \qquad \text{and} \qquad a_{k:m}, \tag{5.40}$$

whose length is $m - k + 1$. Hence, the computational cost of inserting a single row is

$$\sum_{k=1}^{m} O(m - k + 1) = O(m^2). \tag{5.41}$$

Since X^T contains n rows, processing all data rows requires

$$O(nm^2) \tag{5.42}$$

operations. Applying the same two-dimensional transformations to the right-hand side costs only $O(nm)$ additional operations and is therefore lower order. The final triangular solve $Rw = c$ requires $O(m^2)$ operations. The total computational cost is consequently

$$O(nm^2 + m^2), \tag{5.43}$$

which reduces to

$$O(nm^2) \tag{5.44}$$

for the complete factorization and solve. In the ML-CUP setting, where $n \ll m$ and n is regarded as fixed, this is quadratic in m. This should be contrasted with the generic dense QR cost derived in Section 5.1.3,

$$O((n+m)m^2) = O(nm^2 + m^3). \tag{5.45}$$

The structured implementation removes the cubic $O(m^3)$ term of a generic dense QR factorization by exploiting the already triangular block λI_m and processing only the n dense rows of X^T. Its total computational cost is therefore

$$O(nm^2). \tag{5.46}$$

Since n is fixed to 12 in our project setting, the resulting complexity is

$$O(m^2), \tag{5.47}$$

as required by the project specifications.

The dominant storage cost is the upper triangular factor $R \in \mathbb{R}^{m\times m}$, requiring $O(m^2)$ memory, while the working vectors require only $O(m)$ additional storage. If all Householder reflectors are stored, an additional $O(nm)$ memory is required.

Finally, since the method relies only on orthogonal transformations, it retains the numerical stability of QR factorization and avoids explicitly forming A^TA, whose condition number satisfies $\kappa(A^TA) = \kappa(A)^2$.

Appendix A

Proofs

Lemma 1 ($\hat{X}$ has full column rank)**.** *If $\lambda > 0$, then $\hat{X}^T\hat{X} = XX^T + \lambda^2 I_m \succ 0$.*

Proof. For any nonzero $v \in \mathbb{R}^m$: $v^T(XX^T + \lambda^2 I_m)v = \left\|X^T v\right\|^2 + \lambda^2 \|v\|^2 \geq \lambda^2 \|v\|^2 > 0$. □

Lemma 2 (Eigenvalue shift)**.** *If α is an eigenvalue of A, then $\alpha + c$ is an eigenvalue of $A + cI$.*

Proof. $Av = \alpha v \iff (A + cI)v = (\alpha + c)v$. □

Lemma 3 (Singular values of XX^T)**.** *The eigenvalues of $XX^T \in \mathbb{R}^{m\times m}$ are $\{\sigma_1^2, \ldots, \sigma_n^2, 0, \ldots, 0\}$, where $\sigma_1, \ldots, \sigma_n$ are the singular values of X. In particular, $\lambda_m(XX^T) = 0$ when $m > n$.*

Proof. Let $X = U\Sigma V^T$ be the SVD. Then $XX^T = U\Sigma\Sigma^T U^T$, where $\Sigma\Sigma^T = \operatorname{diag}(\sigma_1^2, \ldots, \sigma_n^2, 0, \ldots, 0) \in \mathbb{R}^{m\times m}$. □

Bibliography

[1] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. 2nd. Springer, 2006.

[2] Jonathan Barzilai and Jonathan M. Borwein. "Two-point step size gradient methods". In: *IMA Journal of Numerical Analysis* 8.1 (1988), pp. 141–148.

[3] Dong C. Liu and Jorge Nocedal. "On the limited memory BFGS method for large scale optimization". In: *Mathematical Programming* 45.1 (1989), pp. 503–528.

[4] Lloyd N. Trefethen and David Bau III. *Numerical Linear Algebra*. SIAM, 1997.

[5] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. 4th. Johns Hopkins University Press, 2013.